Lecture 1: Introduction to Evolutionary Algorithms

Introduction to Evolutionary Computation and Population-Based Search

Course: Evolutionary Algorithms

Prepared for classroom teaching

Instructor: Engr. Muhammad Farooq

Lecture roadmap and learning objectives

Lecture roadmap and learning objectives

Lecture flow

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow
6. Python simulation and interpretation

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow
6. Python simulation and interpretation
7. Wrap-up and discussion

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow
6. Python simulation and interpretation
7. Wrap-up and discussion

By the end of the lecture, students should be able to:

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow
6. Python simulation and interpretation
7. Wrap-up and discussion

By the end of the lecture, students should be able to:

- explain an optimization problem

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow
6. Python simulation and interpretation
7. Wrap-up and discussion

By the end of the lecture, students should be able to:

- explain an optimization problem
- describe search in a solution space

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow
6. Python simulation and interpretation
7. Wrap-up and discussion

By the end of the lecture, students should be able to:

- explain an optimization problem
- describe search in a solution space
- define evolutionary algorithms at a high level

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow
6. Python simulation and interpretation
7. Wrap-up and discussion

By the end of the lecture, students should be able to:

- explain an optimization problem
- describe search in a solution space
- define evolutionary algorithms at a high level
- identify core EA components

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow
6. Python simulation and interpretation
7. Wrap-up and discussion

By the end of the lecture, students should be able to:

- explain an optimization problem
- describe search in a solution space
- define evolutionary algorithms at a high level
- identify core EA components
- compare random search, hill climbing, and evolutionary search

Lecture roadmap and learning objectives

Lecture flow

1. Motivation and context
2. What is optimization?
3. Why classical methods may fail
4. What are evolutionary algorithms?
5. Core EA workflow
6. Python simulation and interpretation
7. Wrap-up and discussion

By the end of the lecture, students should be able to:

- explain an optimization problem
- describe search in a solution space
- define evolutionary algorithms at a high level
- identify core EA components
- compare random search, hill climbing, and evolutionary search
- interpret a simple simulation

Motivation: from observed processes to computation

Opening questions

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

- variation

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

- variation
- inheritance

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

- variation
- inheritance
- competition

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

- variation
- inheritance
- competition
- survival of better solutions over time

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

- variation
- inheritance
- competition
- survival of better solutions over time

Why this matters in practice

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

- variation
- inheritance
- competition
- survival of better solutions over time

Why this matters in practice

- huge search spaces

Key message: Evolutionary algorithms are useful when exact methods are too expensive and near-optimal solutions are acceptable.

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

- variation
- inheritance
- competition
- survival of better solutions over time

Why this matters in practice

- huge search spaces
- irregular or noisy objectives

Key message: Evolutionary algorithms are useful when exact methods are too expensive and near-optimal solutions are acceptable.

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

- variation
- inheritance
- competition
- survival of better solutions over time

Why this matters in practice

- huge search spaces
- irregular or noisy objectives
- no gradient information

Key message: Evolutionary algorithms are useful when exact methods are too expensive and near-optimal solutions are acceptable.

Motivation: from observed processes to computation

Opening questions

- What observed processes lead to highly adapted organisms over time?
- Can we use a similar idea to solve engineering and computing problems?

Observed adaptive processes include

- variation
- inheritance
- competition
- survival of better solutions over time

Why this matters in practice

- huge search spaces
- irregular or noisy objectives
- no gradient information
- many good trade-offs instead of one exact answer

Key message: Evolutionary algorithms are useful when exact methods are too expensive and near-optimal solutions are acceptable.

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer
- **Search space**: set of all possible candidates

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer
- **Search space**: set of all possible candidates
- **Objective / fitness function**: measures quality

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer
- **Search space**: set of all possible candidates
- **Objective / fitness function**: measures quality
- **Constraints**: rules that valid solutions must satisfy

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer
- **Search space**: set of all possible candidates
- **Objective / fitness function**: measures quality
- **Constraints**: rules that valid solutions must satisfy

Examples

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer
- **Search space**: set of all possible candidates
- **Objective / fitness function**: measures quality
- **Constraints**: rules that valid solutions must satisfy

Examples

- route between cities

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer
- **Search space**: set of all possible candidates
- **Objective / fitness function**: measures quality
- **Constraints**: rules that valid solutions must satisfy

Examples

- route between cities
- a set of machine settings

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer
- **Search space**: set of all possible candidates
- **Objective / fitness function**: measures quality
- **Constraints**: rules that valid solutions must satisfy

Examples

- route between cities
- a set of machine settings
- a list of selected features

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer
- **Search space**: set of all possible candidates
- **Objective / fitness function**: measures quality
- **Constraints**: rules that valid solutions must satisfy

Examples

- route between cities
- a set of machine settings
- a list of selected features
- class timetable or schedule

What is optimization?

Definition

An optimization problem asks: *among all possible solutions, which one is best according to some criterion?*

Core components

- **Candidate solution**: one possible answer
- **Search space**: set of all possible candidates
- **Objective / fitness function**: measures quality
- **Constraints**: rules that valid solutions must satisfy

Examples

- route between cities
- a set of machine settings
- a list of selected features
- class timetable or schedule

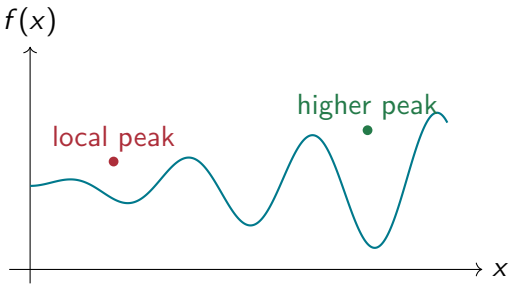
Typical goals

Minimize cost, distance, or time **or** Maximize profit, accuracy, or performance

A simple optimization example

Consider the function

$$f(x) = x \sin(10\pi x) + 1, \qquad x \in [-1, 2]$$

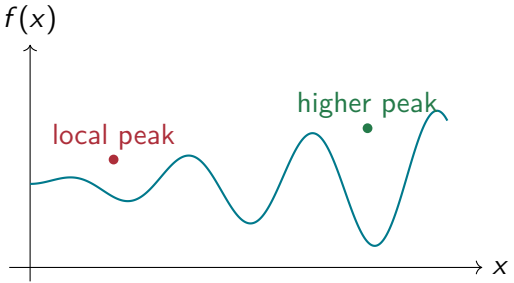


A simple optimization example

Consider the function

$$f(x) = x \sin(10\pi x) + 1, \qquad x \in [-1, 2]$$

Interpretation



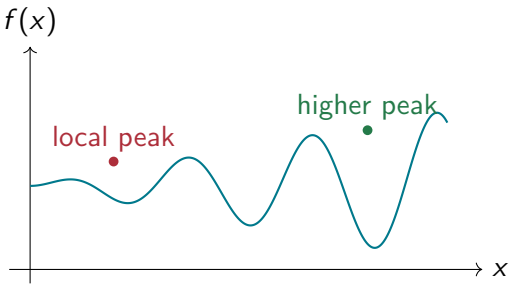
A simple optimization example

Consider the function

$$f(x) = x \sin(10\pi x) + 1, \qquad x \in [-1, 2]$$

Interpretation

- each value of x is a candidate solution



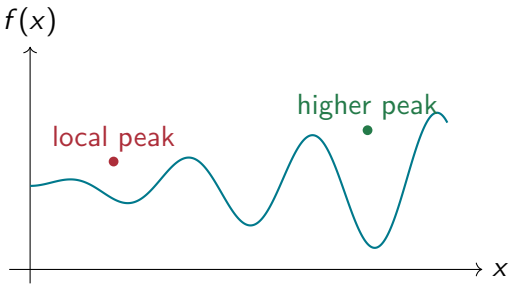
A simple optimization example

Consider the function

$$f(x) = x \sin(10\pi x) + 1, \qquad x \in [-1, 2]$$

Interpretation

- each value of x is a candidate solution
- the interval $[-1, 2]$ is the search space



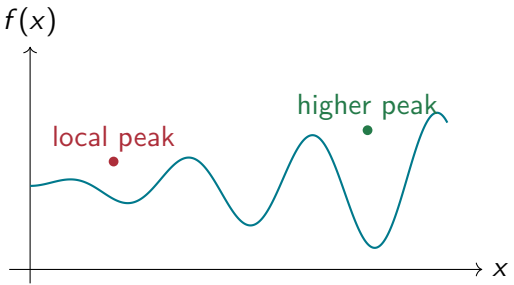
A simple optimization example

Consider the function

$$f(x) = x \sin(10\pi x) + 1, \qquad x \in [-1, 2]$$

Interpretation

- each value of x is a candidate solution
- the interval $[-1, 2]$ is the search space
- $f(x)$ tells us how good a solution is



A simple optimization example

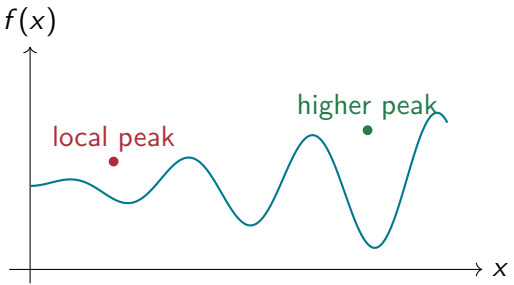
Consider the function

$$f(x) = x \sin(10\pi x) + 1, \qquad x \in [-1, 2]$$

Interpretation

- each value of x is a candidate solution
- the interval $[-1, 2]$ is the search space
- $f(x)$ tells us how good a solution is

Why this is difficult



A simple optimization example

Consider the function

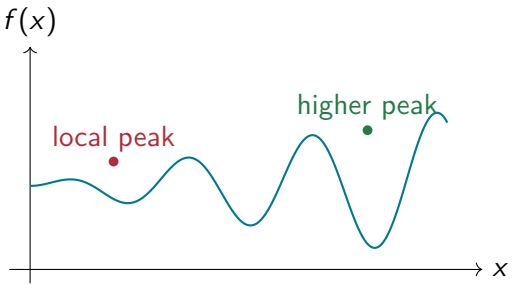
$$f(x) = x \sin(10\pi x) + 1, \qquad x \in [-1, 2]$$

Interpretation

- each value of x is a candidate solution
- the interval $[-1, 2]$ is the search space
- $f(x)$ tells us how good a solution is

Why this is difficult

- the function has many peaks



A simple optimization example

Consider the function

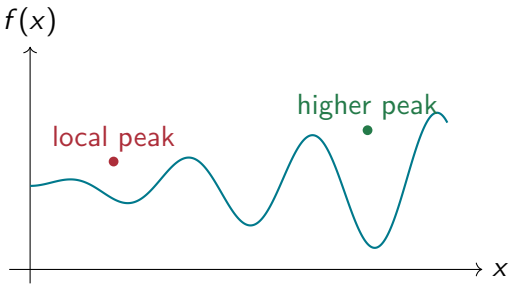
$$f(x) = x \sin(10\pi x) + 1, \qquad x \in [-1, 2]$$

Interpretation

- each value of x is a candidate solution
- the interval $[-1, 2]$ is the search space
- $f(x)$ tells us how good a solution is

Why this is difficult

- the function has many peaks
- some peaks are only locally good



A simple optimization example

Consider the function

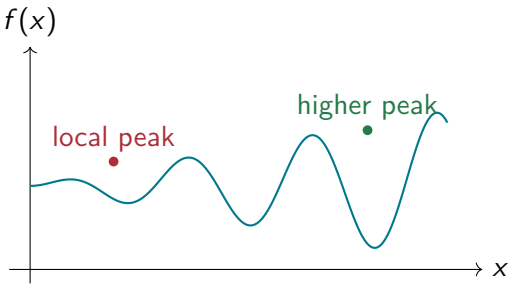
$$f(x) = x \sin(10\pi x) + 1, \qquad x \in [-1, 2]$$

Interpretation

- each value of x is a candidate solution
- the interval $[-1, 2]$ is the search space
- $f(x)$ tells us how good a solution is

Why this is difficult

- the function has many peaks
- some peaks are only locally good
- a search method may stop too early



Why not just use classical methods?

Why not just use classical methods?

Why not just use classical methods?

Exhaustive search

Check every possible solution.

Problem: impossible when the search space is huge.

$$2^{100}$$

possible binary strings of length 100.

Why not just use classical methods?

Exhaustive search

Check every possible solution.

Problem: impossible when the search space is huge.

$$2^{100}$$

possible binary strings of length 100.

Takeaway: classical methods can be powerful, but each has assumptions that break on hard search landscapes.

Why not just use classical methods?

Exhaustive search

Check every possible solution.

Problem: impossible when the search space is huge.

$$2^{100}$$

possible binary strings of length 100.

Gradient methods

Useful for smooth, differentiable functions.

Problem: many real problems are discrete, noisy, discontinuous, or multimodal.

Takeaway: classical methods can be powerful, but each has assumptions that break on hard search landscapes.

Why not just use classical methods?

Exhaustive search

Check every possible solution.

Problem: impossible when the search space is huge.

$$2^{100}$$

possible binary strings of length 100.

Gradient methods

Useful for smooth, differentiable functions.

Problem: many real problems are discrete, noisy, discontinuous, or multimodal.

Takeaway: classical methods can be powerful, but each has assumptions that break on hard search landscapes.

Why not just use classical methods?

Exhaustive search

Check every possible solution.

Problem: impossible when the search space is huge.

$$2^{100}$$

possible binary strings of length 100.

Gradient methods

Useful for smooth, differentiable functions.

Problem: many real problems are discrete, noisy, discontinuous, or multimodal.

Hill climbing

Move from one solution to a better neighbor.

Problem: can get stuck in local optima.

Takeaway: classical methods can be powerful, but each has assumptions that break on hard search landscapes.

Local optimum vs global optimum

Transition point: Instead of exploring with one point only, evolutionary methods maintain *many* candidate solutions at once.

Local optimum vs global optimum

Definitions

Analogy A climber on a foggy mountain may reach the top of a small hill and think it is the highest point, even though a taller mountain exists elsewhere.

Transition point: Instead of exploring with one point only, evolutionary methods maintain *many* candidate solutions at once.

Local optimum vs global optimum

Definitions

- **Local optimum**: better than nearby solutions

Analogy A climber on a foggy mountain may reach the top of a small hill and think it is the highest point, even though a taller mountain exists elsewhere.

Transition point: Instead of exploring with one point only, evolutionary methods maintain *many* candidate solutions at once.

Local optimum vs global optimum

Definitions

- **Local optimum**: better than nearby solutions
- **Global optimum**: best among all feasible solutions

Analogy A climber on a foggy mountain may reach the top of a small hill and think it is the highest point, even though a taller mountain exists elsewhere.

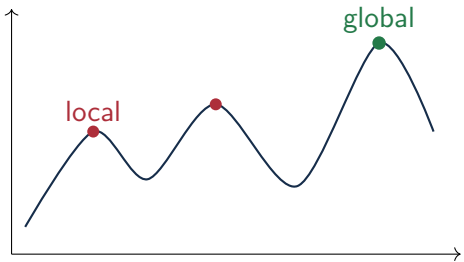
Transition point: Instead of exploring with one point only, evolutionary methods maintain *many* candidate solutions at once.

Local optimum vs global optimum

Definitions

- **Local optimum**: better than nearby solutions
- **Global optimum**: best among all feasible solutions

Analogy A climber on a foggy mountain may reach the top of a small hill and think it is the highest point, even though a taller mountain exists elsewhere.



Transition point: Instead of exploring with one point only, evolutionary methods maintain *many* candidate solutions at once.

What are evolutionary algorithms?

Definition

Evolutionary algorithms are optimization methods inspired by observed patterns in biological evolution.

What are evolutionary algorithms?

Definition

Evolutionary algorithms are optimization methods inspired by observed patterns in biological evolution.

They repeatedly improve a **population** of candidate solutions using:

What are evolutionary algorithms?

Definition

Evolutionary algorithms are optimization methods inspired by observed patterns in biological evolution.

They repeatedly improve a **population** of candidate solutions using:

Selection

What are evolutionary algorithms?

Definition

Evolutionary algorithms are optimization methods inspired by observed patterns in biological evolution.

They repeatedly improve a **population** of candidate solutions using:

What are evolutionary algorithms?

Definition

Evolutionary algorithms are optimization methods inspired by observed patterns in biological evolution.

They repeatedly improve a **population** of candidate solutions using:

What are evolutionary algorithms?

Definition

Evolutionary algorithms are optimization methods inspired by observed patterns in biological evolution.

They repeatedly improve a **population** of candidate solutions using:

Main idea Better solutions are more likely to influence future generations.

What are evolutionary algorithms?

Definition

Evolutionary algorithms are optimization methods inspired by observed patterns in biological evolution.

They repeatedly improve a **population** of candidate solutions using:

Main idea Better solutions are more likely to influence future generations.

Important note We do *not* simulate biology exactly; we borrow useful search principles from observed adaptive processes.

Biological inspiration vs algorithmic abstraction

Biology	Algorithm
organism	candidate solution
population	set of candidate solutions
environment	optimization problem
fitness	quality of a solution
reproduction	creating new solutions
mutation	random change
natural selection	choosing better solutions

Core EA vocabulary

Core EA vocabulary

Core EA vocabulary

- **Population**: collection of candidate solutions

Core EA vocabulary

- **Population**: collection of candidate solutions
- **Individual**: one candidate solution

Core EA vocabulary

- **Population**: collection of candidate solutions
- **Individual**: one candidate solution
- **Fitness**: score for solution quality

Core EA vocabulary

- **Population**: collection of candidate solutions
- **Individual**: one candidate solution
- **Fitness**: score for solution quality
- **Selection**: choosing better individuals to reproduce

Core EA vocabulary

- **Population**: collection of candidate solutions
- **Individual**: one candidate solution
- **Fitness**: score for solution quality
- **Selection**: choosing better individuals to reproduce

Core EA vocabulary

- **Population**: collection of candidate solutions
- **Individual**: one candidate solution
- **Fitness**: score for solution quality
- **Selection**: choosing better individuals to reproduce
- **Crossover / recombination**: combine information from parents

Core EA vocabulary

- **Population**: collection of candidate solutions
- **Individual**: one candidate solution
- **Fitness**: score for solution quality
- **Selection**: choosing better individuals to reproduce
- **Crossover / recombination**: combine information from parents
- **Mutation**: apply random changes

Core EA vocabulary

- **Population**: collection of candidate solutions
- **Individual**: one candidate solution
- **Fitness**: score for solution quality
- **Selection**: choosing better individuals to reproduce
- **Crossover / recombination**: combine information from parents
- **Mutation**: apply random changes
- **Generation**: one evaluation–reproduction cycle

Core EA vocabulary

- **Population**: collection of candidate solutions
- **Individual**: one candidate solution
- **Fitness**: score for solution quality
- **Selection**: choosing better individuals to reproduce
- **Crossover / recombination**: combine information from parents
- **Mutation**: apply random changes
- **Generation**: one evaluation–reproduction cycle
- **Termination**: rule for stopping the algorithm

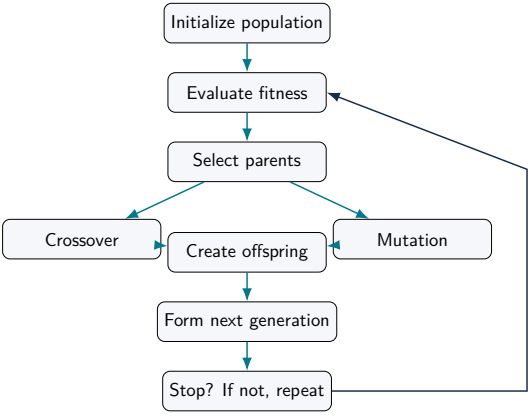
Core EA vocabulary

- **Population**: collection of candidate solutions
- **Individual**: one candidate solution
- **Fitness**: score for solution quality
- **Selection**: choosing better individuals to reproduce
- **Crossover / recombination**: combine information from parents
- **Mutation**: apply random changes
- **Generation**: one evaluation–reproduction cycle
- **Termination**: rule for stopping the algorithm

Common stopping criteria

Fixed number of generations, target fitness reached, or no improvement for many generations.

General workflow of an evolutionary algorithm



Important insight: Evolutionary algorithms are *stochastic*, *iterative*, and *approximate*—they do not guarantee the perfect solution every time, but they are often highly effective.

Random search vs hill climbing vs evolutionary search

Method	Main idea	Advantage	Limitation
Random search	Sample solutions randomly and keep the best	Very simple and easy to implement	Does not learn from previous good solutions
Hill climbing	Start with one solution and move to better neighbors	Often fast on easy landscapes	Easily trapped in local optima
Evolutionary algorithms	Maintain a population and improve it over generations	Better exploration and more robust search	More parameters and computational cost

Python simulation: objective function

```python
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x * np.sin(10 * np.pi * x) + 1
```

Idea: We use a multimodal function with many peaks, so different search methods behave differently.

Python simulation: random search and hill climbing

```python
def random_search(n_samples=100):
    xs = np.random.uniform(-1, 2, n_samples)
    ys = f(xs)
    best_idx = np.argmax(ys)
    return xs, ys, xs[best_idx], ys[best_idx]

def hill_climb(start_x, step_size=0.05, max_iters=100):
    x = start_x
    history = [(x, f(x))]

    for _ in range(max_iters):
        candidates = [c for c in [x-step_size, x+step_size]
                      if -1 <= c <= 2]
        best_candidate = max(candidates, key=f)

        if f(best_candidate) > f(x):
            x = best_candidate
            history.append((x, f(x)))
        else:
            break

    return history
```

Python simulation: population-based sampling

```python
def population_sampling(pop_size=20, generations=20):
    best_history, avg_history = [], []
    population = np.random.uniform(-1, 2, pop_size)

    for _ in range(generations):
        fitness = f(population)
        best_history.append(np.max(fitness))
        avg_history.append(np.mean(fitness))

        idx = np.argsort(fitness)[::-1]
        survivors = population[idx[:pop_size // 2]]

        offspring = []
        while len(offspring) < pop_size:
            parent = np.random.choice(survivors)
            child = parent + np.random.normal(0, 0.1)
            child = np.clip(child, -1, 2)
            offspring.append(child)

        population = np.array(offspring)

    return best_history, avg_history, population
```

Python simulation: running the methods

```
np.random.seed(42)

rs_xs, rs_ys, rs_best_x, rs_best_y = random_search(n_samples=100)

start_x = np.random.uniform(-1, 2)
hc_history = hill_climb(start_x=start_x, step_size=0.05, max_iters=100)

best_hist, avg_hist, final_pop = population_sampling(
    pop_size=20, generations=20
)
print("Random Search Final x =", rs_best_x, "Final fitness =", rs_best_y)
print("Hill Climbing Final x =", hc_history[-1][0], "Final fitness =", hc_history
    [-1][1])
print("Population Sampling Final x =", final_pop[np.argmax(f(final_pop))],
      "Final fitness =", np.max(f(final_pop)))
```

Short conceptual examples for class discussion

Short conceptual examples for class discussion

Short conceptual examples for class discussion

Timetable scheduling

How many ways can courses be assigned to rooms and times?

The number grows extremely quickly, so heuristic search becomes useful.

Short conceptual examples for class discussion

Timetable scheduling

How many ways can courses be assigned to rooms and times?

The number grows extremely quickly, so heuristic search becomes useful.

Short conceptual examples for class discussion

Timetable scheduling

How many ways can courses be assigned to rooms and times?

The number grows extremely quickly, so heuristic search becomes useful.

Feature selection

With 50 features, the number of subsets is

$$2^{50}$$

Too large for brute force.

Short conceptual examples for class discussion

Timetable scheduling

How many ways can courses be assigned to rooms and times?

The number grows extremely quickly, so heuristic search becomes useful.

Feature selection

With 50 features, the number of subsets is

$$2^{50}$$

Too large for brute force.

Short conceptual examples for class discussion

Timetable scheduling

How many ways can courses be assigned to rooms and times?

The number grows extremely quickly, so heuristic search becomes useful.

Feature selection

With 50 features, the number of subsets is

$$2^{50}$$

Too large for brute force.

Engineering design

If 10 parameters each have 100 possible values, then

$$100^{10}$$

possible designs must be considered.

Discussion questions and recap

Discussion questions and recap

Discussion questions

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?
- What role does randomness play in an evolutionary algorithm?

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?
- What role does randomness play in an evolutionary algorithm?
- Why might a population lose diversity over time?

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?
- What role does randomness play in an evolutionary algorithm?
- Why might a population lose diversity over time?
- Can an evolutionary algorithm guarantee the global optimum?

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?
- What role does randomness play in an evolutionary algorithm?
- Why might a population lose diversity over time?
- Can an evolutionary algorithm guarantee the global optimum?

Expected answers

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?
- What role does randomness play in an evolutionary algorithm?
- Why might a population lose diversity over time?
- Can an evolutionary algorithm guarantee the global optimum?

Expected answers

- A local method only checks nearby solutions

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?
- What role does randomness play in an evolutionary algorithm?
- Why might a population lose diversity over time?
- Can an evolutionary algorithm guarantee the global optimum?

Expected answers

- A local method only checks nearby solutions
- Multiple solutions help explore different regions

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?
- What role does randomness play in an evolutionary algorithm?
- Why might a population lose diversity over time?
- Can an evolutionary algorithm guarantee the global optimum?

Expected answers

- A local method only checks nearby solutions
- Multiple solutions help explore different regions
- Randomness supports exploration and variation

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?
- What role does randomness play in an evolutionary algorithm?
- Why might a population lose diversity over time?
- Can an evolutionary algorithm guarantee the global optimum?

Expected answers

- A local method only checks nearby solutions
- Multiple solutions help explore different regions
- Randomness supports exploration and variation
- Strong selection can reduce diversity over time

Discussion questions and recap

Discussion questions

- Why can hill climbing get stuck even when better solutions exist elsewhere?
- Why can keeping multiple solutions be better than keeping only one?
- What role does randomness play in an evolutionary algorithm?
- Why might a population lose diversity over time?
- Can an evolutionary algorithm guarantee the global optimum?

Expected answers

- A local method only checks nearby solutions
- Multiple solutions help explore different regions
- Randomness supports exploration and variation
- Strong selection can reduce diversity over time
- Evolutionary algorithms do not guarantee the global optimum

Summary and homework

Key ideas from Lecture 1

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.
- Real-world search spaces are often large and difficult.

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.
- Real-world search spaces are often large and difficult.
- Classical methods may fail because of local optima or non-smooth objectives.

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.
- Real-world search spaces are often large and difficult.
- Classical methods may fail because of local optima or non-smooth objectives.
- Evolutionary algorithms improve a *population* of candidate solutions.

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.
- Real-world search spaces are often large and difficult.
- Classical methods may fail because of local optima or non-smooth objectives.
- Evolutionary algorithms improve a *population* of candidate solutions.
- They rely on selection, variation, and iterative improvement.

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.
- Real-world search spaces are often large and difficult.
- Classical methods may fail because of local optima or non-smooth objectives.
- Evolutionary algorithms improve a *population* of candidate solutions.
- They rely on selection, variation, and iterative improvement.
- They are especially useful for complex, black-box, or multimodal problems.

Mini-assignment

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.
- Real-world search spaces are often large and difficult.
- Classical methods may fail because of local optima or non-smooth objectives.
- Evolutionary algorithms improve a *population* of candidate solutions.
- They rely on selection, variation, and iterative improvement.
- They are especially useful for complex, black-box, or multimodal problems.

Mini-assignment

- increase the number of random-search samples

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.
- Real-world search spaces are often large and difficult.
- Classical methods may fail because of local optima or non-smooth objectives.
- Evolutionary algorithms improve a *population* of candidate solutions.
- They rely on selection, variation, and iterative improvement.
- They are especially useful for complex, black-box, or multimodal problems.

Mini-assignment

- increase the number of random-search samples
- try different hill-climbing step sizes

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.
- Real-world search spaces are often large and difficult.
- Classical methods may fail because of local optima or non-smooth objectives.
- Evolutionary algorithms improve a *population* of candidate solutions.
- They rely on selection, variation, and iterative improvement.
- They are especially useful for complex, black-box, or multimodal problems.

Mini-assignment

- increase the number of random-search samples
- try different hill-climbing step sizes
- change mutation noise in population sampling

Summary and homework

Key ideas from Lecture 1

- Optimization means finding the best solution in a search space.
- Real-world search spaces are often large and difficult.
- Classical methods may fail because of local optima or non-smooth objectives.
- Evolutionary algorithms improve a *population* of candidate solutions.
- They rely on selection, variation, and iterative improvement.
- They are especially useful for complex, black-box, or multimodal problems.

Mini-assignment

- increase the number of random-search samples
- try different hill-climbing step sizes
- change mutation noise in population sampling
- run the program multiple times and compare outcomes

Next lecture: chromosome representation, selection, crossover, mutation, and the full genetic algorithm cycle.